
Angular Assignment – Core Concepts

Instructions for Students

- Use **Angular 20+**.
 - Follow **best practices** (component-based design, clean templates).
 - Use **TypeScript strictly**.
 - No backend required; use in-memory data.
 - Each question should be implemented as a **separate component** unless mentioned otherwise.
-

Part 1: Angular Data Binding

Q1. Interpolation Binding

Create a component user-profile that:

- Displays **User Name**, **Email**, and **Role** using interpolation.
- Values should be defined in the component class.

Expected Concepts

- `{{ }}` interpolation
 - Component property binding
-

Q2. Property Binding

Create an image gallery component where:

- Image URL is bound using **property binding**
- Image width and height are controlled from the component class

Expected Concepts

- `[src]`
- `[width]`, `[height]`

Q3. Event Binding

Create a counter component that:

- Has **Increment** and **Decrement** buttons
- Updates the count on button click

Expected Concepts

- (click) event binding
 - Component methods
-

Q4. Two-Way Binding

Create a form field where:

- User enters their **name**
- The name is displayed live below the input

Expected Concepts

- [(ngModel)]
 - FormsModule import
-

Q5. Binding Comparison

Explain with code:

- Difference between **Interpolation** and **Property Binding**
 - Difference between **Event Binding** and **Two-Way Binding**
-

Part 2: Angular Directives

Q6. Structural Directive – *ngIf

Create a login status component:

- Show **“Welcome User”** if isLoggedIn = true

- Show “**Please Login**” otherwise

Expected Concepts

- *ngIf
 - Boolean conditions
-

Q7. Structural Directive – *ngFor

Create a product list:

- Display a list of products with **name and price**
- Data should come from an array in the component

Expected Concepts

- *ngFor
 - Index usage
-

Q8. Attribute Directive – ngClass

Create a task list where:

- Completed tasks appear in **green**
- Pending tasks appear in **red**

Expected Concepts

- [ngClass]
 - Conditional CSS classes
-

Q9. Attribute Directive – ngStyle

Create a paragraph whose:

- Font size and color change dynamically using buttons

Expected Concepts

- [ngStyle]

- Dynamic styling
-

Q10. Custom Directive (Optional – Advanced)

Create a custom directive:

- Highlights text background when mouse hovers

Expected Concepts

- @Directive
 - @HostListener
 - @HostBinding
-

Part 3: Angular Forms

Q11. Template-Driven Form – Basic

Create a **Registration Form** with:

- Name
- Email
- Password
- Submit button

Expected Concepts

- ngForm
 - ngModel
 - Form submission
-

Q12. Template-Driven Form – Validation

Enhance the registration form:

- Name is required
- Email must be valid

- Show validation error messages

Expected Concepts

- required
 - email
 - Form state (touched, invalid)
-

Q13. Reactive Form – Basic

Create a **Login Form** using Reactive Forms:

- Email
- Password
- Submit button

Expected Concepts

- FormGroup
 - FormControl
 - ReactiveFormsModule
-

Q14. Reactive Form – Validation

Add validations:

- Email required and valid
- Password minimum 6 characters
- Disable submit button if form is invalid

Expected Concepts

- Validators
 - Form status checks
-

Q15. Reactive Form – Dynamic Control

Create a form where:

- User can **add/remove phone numbers dynamically**

Expected Concepts

- FormArray
 - Dynamic form controls
-

Part 4: Scenario-Based Assignment

Q16. Mini Use Case

Create a **Student Management UI**:

- Add student using a form
- Display students in a table
- Highlight failed students using directives
- Use two-way binding for search filter

Must Use

- Data binding
 - Directives
 - Forms
-